

Enhancing Embedding Representation Stability in Recommendation Systems with Semantic ID

Carolina Zheng^{*†1}, Minhui Huang[†], Dmitrii Pedchenko[†], Kaushik Rangadurai[†], Siyu Wang[†], Gaby Nahum[†], Jie Lei[†], Yang Yang[†], Tao Liu[†], Zutian Luo[†], Xiaohan Wei[†], Dinesh Ramasamy[†], Jiyan Yang[†], Yiping Han[†], Lin Yang[†], Hangjun Xu[†], Rong Jin[†], Shuang Yang[†]

^{*}Columbia University, [†]AI at Meta

¹Work done during 2024 Internship at Meta

The exponential growth of online content has posed significant challenges to ID-based models in industrial recommendation systems, ranging from extremely high cardinality and dynamically growing ID space, to highly skewed engagement distributions, to prediction instability as a result of natural id life cycles (e.g, the birth of new IDs and retirement of old IDs). To address these issues, many systems rely on random hashing to handle the id space and control the corresponding model parameters (i.e embedding table). However, this approach introduces data pollution from multiple ids sharing the same embedding, leading to degraded model performance and embedding representation instability.

This paper examines these challenges and introduces Semantic ID prefix ngram, a novel token parameterization technique that significantly improves the performance of the original Semantic ID. Semantic ID prefix ngram creates semantically meaningful collisions by hierarchically clustering items based on their content embeddings, as opposed to random assignments. Through extensive experimentation, we demonstrate that Semantic ID prefix ngram not only addresses embedding instability but also significantly improves tail id modeling, reduces overfitting, and mitigates representation shifts. We further highlight the advantages of Semantic ID prefix ngram in attention-based models that contextualize user histories, showing substantial performance improvements. We also report our experience of integrating Semantic ID into Meta production Ads Ranking system, leading to notable performance gains and enhanced prediction stability in live deployments.

Correspondence: cz2539@columbia.edu, {mhhuang, dmitripedchenko, krangadu, siyuw, gnahum12345, jielei, yzyang, tliu97, zutianluo, ubimeteor, dineshr, chocjy, yipinghan, ylin1, rongjinml, hangjunxu, shuangyang}@meta.com

Keywords: Recommendation Systems, Content Understanding, Representation Learning, Vector Quantization 

1 Introduction

Item recommendation can involve many signal-rich features, including categorical features corresponding to item IDs. The raw item IDs are usually mapped to embeddings, which are then further processed by deep learning-based model architectures such as the widely deployed Deep Learning Recommendation Model (DLRM) (Covington et al., 2016; Naumov et al., 2019). However, in industry-scale online settings, several key data-related challenges have emerged in learning item embedding representations. In particular, *item cardinality*, the huge number of total items; *impression skew*, the fact that only a few items comprise most user impressions or conversions (Milojević, 2010); and *ID drifting*, or the majority of items entering and leaving the system within short time periods (Gama et al., 2014).

A popular and simple approach to learning embedding representations is random hashing, where raw item IDs are randomly hashed to share the same embedding (Zhang et al., 2020). Hashing is used due to the large item cardinality and system constraints on embedding table sizes. However, random hashing and ID drifting together lead to undesirable *embedding representation instability* when the model is trained over long time periods. This is due to the nature of random hash collisions, which result in contradictory gradient updates to the embedding weights. Further, as the items in the system change over time from ID drifting, the learning from old items is lost and the embedding weights for new items are essentially random. This approach is ill-suited for items with few impressions, which are the majority of items due to impression skew.

To mitigate these drawbacks, a stable ID space is needed. A stable ID space ideally ensures that a learned embedding representation has a stable meaning as the model learns from more data. In this work, we investigate a recently proposed item representation approach called Semantic ID (Singh et al., 2023; Rajput et al., 2024) as a candidate for a stable ID space. Semantic ID derives item IDs based on hierarchical clusters learned from the semantic similarity of items as given by their text, image, or video content. A given item’s Semantic ID is then mapped to embedding representations via a parameterization scheme. Importantly, the ID space of Semantic ID is fixed *a priori* and has semantic meaning – meaning that it can address embedding representation instability. However, one challenge in using Semantic ID in recommendation modeling is defining a mapping from its cluster assignments to the embedding table.

The main contributions of this paper are:

- Using experiments on a simplified version of Meta’s production ads ranking model, we deepen the empirical understanding of how Semantic ID improves embedding representation stability. We further propose Semantic ID prefix-ngram, a novel token parameterization technique on top of Semantic ID that brings significant performance gains compared to the original Semantic ID introduced in (Singh et al., 2023).
- We characterize the item data distribution in terms of the number of items (item cardinality), the fact that most items have few impressions (impression skew), and the short item lifetime in the system (ID drifting) and explain their

connection with embedding representation stability.

- We describe the productionization of Semantic ID prefix-gram into both sparse and sequential features in Meta’s production system. We show that adding these features brings online performance gains and improves online prediction stability.

In offline experiments on Meta’s ads ranking data, we show that Semantic ID improves generalization and is less sensitive to distribution shift compared to random hashing. Confirming our hypothesis on impression skew, we find that most gains from Semantic ID come from the long tail of the item distribution. We show that by incorporating hierarchical cluster information, the proposed prefix-gram is crucial to Semantic ID’s effectiveness. We also demonstrate that semantic similarity translates to prediction similarity in both online and offline settings (Section 6.3 and Section 7.4). Further, Semantic ID results in outsized gains when incorporated in contextualizing models of users’ item interaction history.

In an online setting, we describe the implementation of Semantic ID prefix-gram features in Meta’s production ads recommendation system, where they serve as the top sparse features by feature importance and result in **0.15% online performance gain**. Finally, we find that incorporating Semantic ID features significantly reduces the model’s prediction variance for the same item. This is crucial to ensure advertisers’ trust in Meta’s recommendation system and to improve stability of the final item ranking.

The remaining sections are organized as follows: Section 2 explains related work. Section 3 provides an overview of the ranking model. Section 4 introduces Semantic ID and token parameterization. Section 5 explains the three item impression distribution challenges. Section 6 describes the offline experiments. Section 7 describes the productionization of Semantic ID at Meta and the online experiments. Section 8 concludes.

2 Related Work

Item representations in recommendation Many modern deep learning recommendation models use trained embeddings to represent categorical (“sparse”) features (Covington et al., 2016; Naumov et al., 2019; Naumov, 2019). A simple solution to high item cardinality is to use random hashing (Weinberger et al., 2009), but random hash collisions can be undesirable. One option is to modify the hashing procedure. Under this category, collision-free hashing (Liu et al., 2022) introduces individual embeddings for each item by dynamically free the memory of embeddings for retired items. Double hashing (Zhang et al., 2020) utilizes two independent hash functions to reduce memory usage, but still has random collision. Learning to hash methods (Wang et al., 2017) focus on similarity preserving by training ML-based hash functions. There have also been works that address impression skew through contrastive learning or clustering (Yao et al., 2021; Chang et al., 2024); we view these as complementary approaches. We take a holistic approach of designing a stable ID space, to minimize the need for hashing and to address embedding representation shifting directly.

Stable embedding representation Stable ID is inspired by tokenization approaches in NLP, which learn a fixed vocabulary of

tokens to represent text in language modeling (Sennrich, 2015; Kudo, 2018; Devlin, 2018). In designing a tokenization scheme for item recommendation, Hou et al. (2023) proposes to vector-quantize the embeddings learned from an item content understanding model; Qu et al. (2024) introduce a masked vector-quantizer to transfer the learned representations from collaborative filtering models to a generative recommender. Semantic ID is introduced concurrently in (Singh et al., 2023; Rajput et al., 2024), which is based on (Hou et al., 2023) and uses an RQ-VAE for quantization, showing its benefits in generalization performance and sequential recommendation, respectively. In this work, we adapt Semantic ID as our stable ID method and analyze its effectiveness in addressing the three challenges in online item recommendation.

3 Ranking Model Overview

The recommendation problem is posed as a classification task, where a data point is the user- and item-side features associated with an item impression or conversion and a binary label indicating whether or not the user interacted or converted for that item. We now give a brief overview of the ranking model architecture.

3.1 Model

The recommendation system follows a deep neural architecture based on the DLRM (Covington et al., 2016; Naumov et al., 2019). The model consists of three stacked sections. First is the information aggregation section, where the sparse (i.e., categorical), dense, and user history-based features are processed independently. The output of each of these modules is a list of embedding vectors. Second, these are concatenated into a single list which goes through the interaction layer, where dot products (or higher order interactions) are taken between all pairs of vectors. Third, the output of the interaction layer is transformed via an MLP to produce the logit score and a sigmoid is taken to output a probability. The model is trained using cross-entropy loss.

In the remainder of the paper we focus on the information aggregation section of the model.

Embedding module Let I be the total number of raw IDs in the system and let $[1..N]$ denote the integers from 1 to N .

The embedding table is a matrix $\mathbf{E} \in \mathbb{R}^{H \times d_m}$, where d_m is the embedding dimension and H is the number of embeddings. Let $f = (f_1, \dots, f_G) : [1..I] \rightarrow [1..H]^G$ be an embedding lookup function that maps a raw ID to G embedding table row indices. Then for each raw ID $x \in [1..I]$, the sparse module looks up embedding rows $\mathbf{e}_{f_1}(x), \dots, \mathbf{e}_{f_G}(x)$ and produces a single output embedding via sum-pooling, $\mathbf{e}_f(x) := \sum_{i=1}^G \mathbf{e}_{f_i}(x)$.

Sparse module A sparse feature is a set $\mathbf{x} := \{x_1, \dots, x_n\}$ of raw IDs. For instance, this could be a set of n product category IDs a given item belongs to. We usually produce a single embedding $\mathbf{e}_f(\mathbf{x})$ by sum-pooling embeddings $\mathbf{e}_f(x_i)$ for constituent raw IDs.

User history module We model a user’s item interaction history as a sequence of sparse features $\mathbf{x}^u := (\mathbf{x}_1^u, \dots, \mathbf{x}_T^u)$ and the corresponding interaction timestamps. When working with these features, there are system constraints due to the number of items

and the sequence length T . We include item interaction history for up to three months, which brings the item cardinality for the model to process to over one billion. It is important for the user history module to contextualize the sequence of features before they are further processed downstream. We describe the architecture below.

First, we use the sparse module to embed each sparse feature $\mathbf{x}_i^u$ and obtain a learned timestamp embedding; the sum is $\mathbf{e}_f^u(\mathbf{x}_i^u)$. Let $\mathbf{X} = [\mathbf{e}_f^u(x_1^u); \dots; \mathbf{e}_f^u(x_T^u)]^\top \in \mathbb{R}^{T \times d_m}$ denote the resulting encoding. We then contextualize this sequence of embeddings via an aggregation module. We use one of the following three aggregation module architectures: Bypass, Transformer, and Pooled Multihead Attention (PMA), which are defined in [Appendix A](#).

3.2 Metrics

Normalized Entropy We measure model performance by normalized entropy (NE), defined as the model cross-entropy divided by the cross-entropy from predicting the data mean frequency of positive labels. The NE equation is

$$\text{NE} = \frac{-\frac{1}{N} \sum_{i=1}^N (y_i \log(p_i) + (1 - y_i) \log(1 - p_i))}{-(p \log(p) + (1 - p) \log(1 - p))}, \quad (1)$$

where N is the number of training examples, $y_i \in \{0, 1\}$ is the label for example i , p_i is the model prediction for example i , and $p = \frac{\sum_{i=1}^N y_i}{N}$. Lower is better.

4 Semantic ID and Parameterizations

The primary motivation for Semantic ID is to design an efficient clustering schema to represent items that allows knowledge sharing between items with shared semantics. Intuitively, if we have hundreds of ads about pizza that different users clicked on, we would want an example involving one of the ads to be informed by the other ads' representations. We craft the design of Semantic ID to potentially address the data-related challenges of *item cardinality*, *impression skew*, and *ID drifting* described in [Section 5](#). Compared to embedding representations based on random clusters, semantics-based representations will likely be more stable over time. Semantics-based clustering will also allow tail items to learn from more training examples. The learning from items that have left the system can also be utilized, and embedding weights for new items do not have to be learned from scratch. We investigate these hypotheses empirically in [Section 6](#).

First, we give an overview of Semantic ID in [Section 4.1](#). We then describe token parameterization in [Section 4.2](#). This step is crucial to incorporate Semantic ID into the recommendation model.

4.1 Overview

Semantic IDs are learned for items in two stages: first, apply a content understanding model to the items' text, image, or video to produce dense content embeddings. Then, train an RQ-VAE ([Zeghidour et al., 2021](#)) on the content embeddings to obtain a vector quantization for each item, which is represented as a sequence of coarse-to-fine discrete codes called the item's Semantic ID.

Let L be the number of layers (i.e., length of the sequence) and K be the codebook size (i.e., number of clusters at each

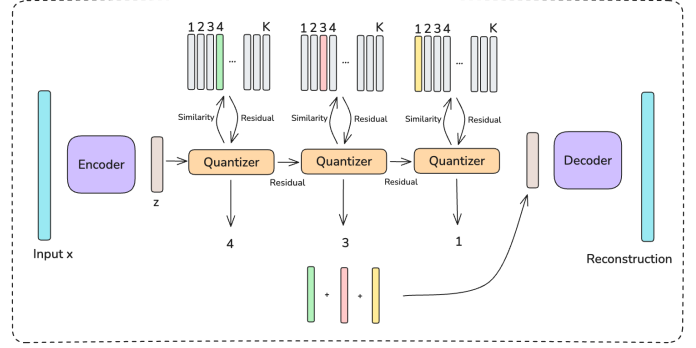


Figure 1 The RQVAE model with $L = 3$.

layer). RQ-VAE consists of an encoder that maps the content embedding $\mathbf{x} \in \mathbb{R}^D$ to a continuous latent representation, $\mathbf{z} \in \mathbb{R}^{D'}$, a residual quantizer that quantizes $\mathbf{z}$ into a series of discrete codes $\mathbf{c} := (c_1, \dots, c_L) \in K^L$, and a decoder that reconstructs $\mathbf{x}$ from $\mathbf{c}$. This is done by associating each layer l with a codebook which is a set of K vectors $\{\mathbf{v}_k^l\}_{k=1}^K$. The sequence of discrete codes is hierarchical: c_l corresponds to the codebook vector $\mathbf{v}_{c_l}^l$ that approximates $\mathbf{r}_l$, the remaining residual from $\mathbf{z}$ after recursively applying the codebook vectors from layers $(l-1)$ to 1, i.e.,

$$\mathbf{r}_l := \mathbf{z} - \sum_{i=1}^{l-1} \mathbf{v}_{c_i}^i, \quad c_l := \arg \min_c \|\mathbf{v}_c^l - \mathbf{r}_l\|_2. \quad (2)$$

In [Section 4.2](#), we provide more intuition on the nature of RQ-VAE's hierarchical clustering and how it informs the choice of token parameterization.

The RQ-VAE is trained using two loss terms, a reconstruction loss and a loss that encourages the residuals and codebook vectors to be close to each other,

$$\begin{aligned} \mathcal{L}_{\text{RQ-VAE}}(\mathbf{x}) &= \|\mathbf{x} - \text{dec}(\mathbf{c})\|^2 \\ &+ \sum_{l=1}^L \beta \|\mathbf{r}_l - \text{sg}(\mathbf{v}_{c_l}^l)\|^2 + \|\text{sg}(\mathbf{r}_l) - \mathbf{v}_{c_l}^l\|^2, \end{aligned}$$

where $\text{dec}(\mathbf{c})$ is the result of applying the decoder to the codes $\mathbf{c}$, $\text{sg}(\cdot)$ corresponds to the stop-gradient operator, and β is a hyperparameter we set to 0.5 in the experiments. A Semantic ID is defined as the sequence of discrete codes $(c_1, \dots, c_L)$ produced by the encoder and residual quantizer.

4.2 Token Parameterization

In our experiments, we use the same codebook size for each level, resulting in K^L total clusters. An important feature of RQ-VAE is that it produces hierarchical clusters. Assuming $L = 3$ for simplicity, a raw item ID is mapped to a sequence (c_1, c_2, c_3) . The precision of vector quantization increases as one moves from the first token, c_1 , to the deeper token c_2 , and finally c_3 . The first token c_1 represents the coarsest bucket: e.g., all ads related to food. The second token c_2 refines this information, e.g., (c_1, c_2) may represent all ads related to pizza. The last token c_3 further refines this information, e.g., (c_1, c_2, c_3) may represent all ads related to pizza and written in a specific language such as English.

Due to this, we can control the amount and the structure of the information that the recommendation model receives from Se-

mantic ID. Notably, supplying the most fine-grained information (all possible $(c_1, c_2, \dots, c_L)$ tuples) is often not feasible due to high cardinality of the possible combinations. Hence, a tradeoff exists between the cardinality of the token parameterization and the amount of information the model receives from Semantic ID.

Table 1 Token parameterization techniques

Token Param	$p(c_1, \dots, c_L; H)$
Trigram	$[K^2 c_1 + K c_2 + c_3]$
Fourgram	$[K^3 c_1 + K^2 c_2 + K c_3 + c_4]$
All bigrams	$[K^2 \times (i - 1) + K c_i + c_{i+1}, \text{ for } i \text{ in } [1 \dots L-1]]$
Prefix-ngram	$[\sum_{t=1}^i K^{i-t} (c_t + 1) - 1, \text{ for } i \text{ in } [1 \dots n]]$

Let $s(x) : [1..I] \rightarrow K^L$ be the Semantic ID lookup that maps raw IDs to Semantic IDs learned by RQ-VAE. Keeping the hierarchical nature of the tokens in mind, we must specify a token parameterization that maps a Semantic ID to embedding table rows, $p(c; H) : K^L \rightarrow [1..H]^G$. Table 1 defines several possible parameterizations. When the Semantic ID cardinality is larger than the embedding size, a modulo hash function is applied. When there are multiple IDs, a shifting factor is added to avoid the collision between different positions. Among all the parameterization techniques, only Prefix-ngram consists of all possible tuples from different granularity.

Table 2 NE performance for different tokenization parameterizations

RQ-VAE $K \times L$	Token Parameterization	Train NE Gain
$[2048] \times 3$	Trigram	-0.028%
$[2048] \times 4$	Fourgram	-0.035%
$[2048] \times 4$	All bigrams	-0.091%
$[512] \times 3$	Prefix-3gram	-0.034%
$[1024] \times 3$	Prefix-3gram	-0.097%
$[2048] \times 3$	Prefix-3gram	-0.141%
$[2048] \times 5$	Prefix-5gram	-0.208%
$[2048] \times 6$	Prefix-6gram	-0.215%

Table 2 summarizes the model performance across different token parameterizations. We draw the following conclusions: i) Prefix-ngram is the best parameterization. This indicates that incorporating the hierarchical nature of the clustering in the embedding table mapping is necessary for effectiveness, as it allows for knowledge sharing among more items than a flat mapping; ii) Increasing the depth of Prefix-ngram improves NE performance; iii) increasing the RQ-VAE cardinality improves NE performance.

5 Item Impression Distribution Issues

In this section, we discuss the data distribution aspects that present challenges for recommendation modeling in Meta ads ranking and how we address them with the use of Semantic ID.

Item cardinality For certain features, such as the target item, the number of distinct items I considered by the model can be much larger than a feasible embedding table size H in the sparse module. In such cases the mapping function $f(\mathbf{x})$ introduces collisions: two or more raw IDs will map to the same row. The mapping function $f(\mathbf{x})$ is often chosen to be a simple hash. Since the initial raw IDs are randomly generated at the time of item

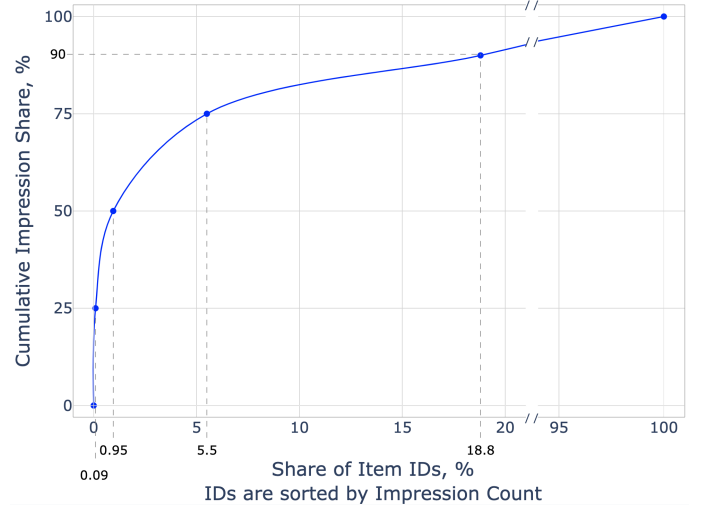


Figure 2 Impression Skew – cumulative impressions as a function of the share of items considered. As items are sorted by the impression count, one sees that the majority of impressions comes from a fraction of most popular items.

creation, the resulting collisions will essentially be random. Such random collisions can negatively affect the resulting representation quality of embeddings and serve as an obstacle to effective knowledge sharing across items.

Impression skew For the target item feature, the item distribution in the training data is highly skewed. Figure 2 shows that in our system, a small percentage of items dominates the item impression distribution: when sorting the items by popularity, the top 0.1% “head” items have 25% of all item impressions, the next 5.5% “torso” items have 50% of cumulative impressions, while the remaining 94.4% “tail” items account for the remaining 25% of impressions.

As tail items have few training examples, it can be challenging to learn embedding representations $\mathbf{e}(\mathbf{x})$ that generalize well. Random hashing doesn’t allow the head and torso items to effectively share knowledge with semantically similar tail items since the assignment of several items to a single embedding is random.

ID drifting The existing item ID space is highly dynamic with large numbers of old items retiring (Figure 3) and new items entering the system. We call this item distribution shift in our system “raw ID drifting.” The raw ID drifting phenomenon stems from the nature of online recommendation systems, where new ads are created on a daily basis and most ads have a relatively short lifespan.

As a byproduct, a recommendation model based on random hashing experiences severe embedding representation drift over time: a given embedding $\mathbf{e}$ represents different items over time as items enter and exit the system.

Item representation with Semantic ID We hypothesize that switching from raw IDs to Semantic IDs can effectively address the issues above.

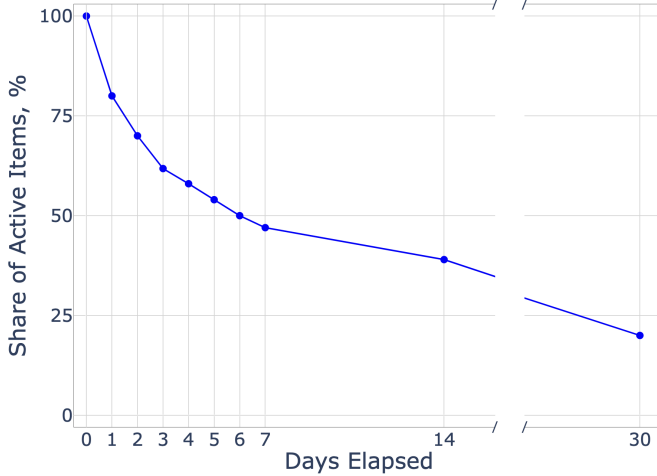


Figure 3 ID Drift – share of items that remain active in the initial corpus as a function of time. Half of the original corpus exits the system after 6 days. An equal number of new items enters the system, creating a severe item distribution drift.

When an advertiser introduces a new ad $\mathbf{x}$ to the system and retires the previous one $\mathbf{y}$, the fine-grained content details of the new ad may be different from the retired one, but the broad semantic category of the product usually remains the same. Therefore, the new and retired ads’ Semantic IDs will match (or at least share a prefix). Hence, the item impression distribution in Semantic ID space exhibits much less drift compared to the raw ID space as long as the broad semantic categories remain temporally stable.

Similarly, if a tail item $\mathbf{x}$ has similar content to a head or torso item $\mathbf{y}$, their Semantic IDs will match (or at least share a prefix). The resulting item impression distribution in Semantic ID space exhibits less skew compared to the raw ID space (see [Appendix B](#)).

In both of the cases above, the embeddings $\mathbf{e}(\mathbf{x})$ and $\mathbf{e}(\mathbf{y})$ will be equal (or similar if the Semantic IDs only share a prefix). This is a way for the model to transfer knowledge from item $\mathbf{y}$ with many training examples to item $\mathbf{x}$. Summarizing, temporal stability of semantic concepts results in stability of Semantic ID encoding, which mitigates embedding representation instability for the model.

6 Offline Experiments

To investigate our hypotheses about the advantages of Semantic ID over baseline item representation approaches, we conduct a suite of offline experiments.

We use a simplified version of Meta’s production ads ranking model, keeping all the dense features and the user’s item interaction history, but including only the target item in the sparse module (and removing $O(100)$ other sparse features). We train on production user interaction data from a four-day time period, processing the training data sequentially and training for a single epoch. We evaluate the model on the first six hours of the next day of data.

6.1 Baselines

In [Section 5](#), we outlined data-related challenges and opportunities in designing a good embedding lookup function $f(\mathbf{x})$ for embedding-based item representations. We describe two baseline approaches, individual embeddings (IE) and random hashing (RH), and compare those to Semantic ID (SemID).

Individual embeddings Each raw ID gets its own embedding table row, $I = H$ and $f_{IE}(x) := x$. While unrealistic in production scenarios due to system constraints, we consider this model for illustration purposes. During evaluation, IDs that are not seen during training are mapped to a randomly initialized untrained embedding.

Random hashing In the case where $I \approx a \cdot H$ for some $a > 1$, we can randomly hash raw IDs to embedding table rows, $f_{RH}(x) := h(x)$, where $h(x) : [1..I] \rightarrow [1..H]$ is one of the standard hash functions such as modulo hash. This creates random collisions with an average collision factor of a .

Semantic ID The items’ content embeddings are obtained from a multimodal image and text foundation model. The foundation model is pre-trained on a large training set of items using image and text alignment objectives ([Radford et al., 2021](#)). The RQ-VAE is then trained on the content embeddings of all target items from the past three months, with $L = 3$ and $K = 2048$. We use the prefix 3-gram parameterization $f_{SemID} = p \circ s$ from [Section 4.2](#).

The analyses in [Section 6.2](#) and [Section 6.3](#) focus on the target item sparse feature. We train three versions of the recommendation model using the above three embedding lookup functions. The size of the item embedding table is equal to the total number of items for IE and set to a smaller size for RH and SemID, with the average collision factor of 3. The user history features are mapped using random hashing.

In [Section 6.4](#), we use Semantic ID for the user history features and study its effect on aggregation module architectures. The item interaction history sequence length is fixed to $O(100)$. We pad or truncate the user history to fit the desired length.

6.2 Segment Analysis

To understand the effect of impression skew on each approach, we segment the data based on the item’s number of impressions in the training period. We sort all items by impression count. As before, we segment items into head, torso, and tail items according to whether they generate 25%, 75%, or 100% of the cumulative impression count in this sorted order. Due to the impression skew, the percentage of items that are head, torso, or tail items are 0.1%, 5.5%, or 94.4%, respectively. We also evaluate on the segment of new items that only appear in the evaluation period and were not seen during training. The performance of the three item representation approaches is shown in [Table 3a](#).

Compared to the baselines, Semantic ID improves generalization for tail items, is NE neutral for head items, and slightly beneficial for torso items. As this is also relative to the individual embeddings approach, Semantic ID is not simply better at clustering than random hashing, but we find that the target item feature benefits from semantics-based knowledge sharing.

Table 3 Performance of three item representation approaches over various item segments.

(a) Evaluation NE (lower is better). Semantic ID enables knowledge transfer to tail and new cold start items.

Cum. Exs.	Item Percentile	Eval NE			SemID NE Gain vs.	
		RH	IE	SemID	RH	IE
25% (Head)	0.1	0.80105	0.80101	0.80108	0.00%	0.01%
75% (Torso)	5.6	0.83589	0.83583	0.83580	−0.01%	−0.00%
100% (Tail)	100	0.83904	0.83886	0.83872	−0.04%	−0.02%
Items Seen in Training		0.82626	0.82612	0.82600	−0.03%	−0.02%
New Items		0.83524	0.83453	0.83180	−0.41%	−0.33%
All Items		0.82663	0.82645	0.82621	−0.05%	−0.03%

(b) Sensitivity to distribution shift:
 $NE[t_0, t_1] - NE[t_0 + 42h, t_1 + 42h]$.
 Lower is better.

Cum. Exs.	RH	IE	SemID
25% (Head)	0.0057	0.0065	0.0059
75% (Torso)	0.0087	0.0075	0.0076
100% (Tail)	0.0128	0.0103	0.0106
All Items	0.0083	0.0074	0.0073

Specifically, the knowledge sharing occurs through the shared embedding weights, which receive updates for semantically-related items. Knowledge sharing benefits are largest on the new items segment, where SemID achieves large gains over both RH and IE (−0.41% and −0.33%, respectively). New items use pre-trained weights from semantically similar items seen during training for prediction, rather than using a non-relevant weight (RH) or an untrained weight (IE).

To measure the effect of embedding representation drifting on model performance, we evaluate the trained models back on the training data but on two different temporal segments. We take NE on 42-48 hours prior to the end of the training epoch and subtract NE on the last six hours of training. A smaller value indicates that the embedding representation shift caused by ID drifting affects model fit less. This is because our model is trained time-sequentially for one epoch, so the resulting model learns to fit the latest training time period at the end of training. The results are in Table 3b.

Individual embeddings approach has a smaller performance gap compared with random hashing. This highlights that random hashing suffers from ID drifting – the embedding representations lose capability to represent older items over time as the weights are updated using new item examples. In contrast, Semantic ID matches the performance of individual embeddings, indicating that its learned representations are more stable over time.

We conjecture that this improved representation stability also allows the model to generalize better over much longer training durations, where ID drifting becomes even more pronounced. We train the RH and SemID models over a 20-day period and compare them to corresponding models trained only for the last four days of the period. The results in Table 4 show that compared to random hashing, the performance of Semantic ID scales better with training data over a longer period.

Table 4 NE improvement from training for 20 days of data instead of 4 days.

	RH	Semantic ID
Eval NE Gain	−0.18%	−0.23%

6.3 Item Representation Space

To gain a better understanding of the item embedding representations, we extract the learned embedding weights from each trained model. One can view random hashing and Semantic ID as two different ways to partition the raw item ID corpus. We

wish to see whether the semantics-based partition produced by Semantic ID is better suited for the recommendation problem than the random partition produced by random hashing.

When several items are assigned to the same partition, they get mapped to the same embedding by the embedding lookup module. We view this embedding vector as a summary of the per-item embeddings learned by the individual embeddings model. While we fit the individual embeddings model for illustration purposes in this paper, IE is impractical in industry-level settings. A partition with lower intra-partition embedding variance and higher inter-partition distances can be viewed as a more effective summary of individual embeddings. We compute these metrics for the RH and SemID partitions of the embeddings learned by the IE model.

We set the collision factor to 5 for this experiment. As a result, the resulting clusters for RH and SemID partitions contain 5 items on average. However, since Semantic ID is the latent codes learned by an RQ-VAE model, the resulting cluster sizes are highly variable. We compute metrics for two groups of Semantic ID clusters, small clusters with 4-10 items each and the top 1,000 clusters, where each cluster contains thousands of items. Table 5 contains the average variances and average pairwise distances, with standard deviations in parentheses. The metrics are averaged across the embedding dimensions to produce single scalars for comparison.

Table 5 Intra- and inter-cluster variances and pairwise distances for random hashing and SemID-based partitions.

	Variance	Pairwise distance
Random Hashing	1.52×10^{-3} (8.0×10^{-4})	0.22 (0.04)
SemID (small)	1.31×10^{-3} (1.0×10^{-3})	0.24 (0.09)
SemID (top 1,000)	1.23×10^{-3} (5.5×10^{-4})	0.06 (0.02)

We observe that the Semantic ID partitions produce clusters with lower intra-cluster variance compared to random hashing. However, the resulting pairwise distances send a mixed signal. We hypothesize that the low pairwise distances between the top 1,000 clusters is because RQ-VAE places multiple centroids into the regions with highest data density to minimize the model loss.

6.4 User History Modeling

In this section, we explore the effect of Semantic ID on user history modeling. One role of the module is to contextualize and summarize the user history.

We find that using Semantic ID and a contextualizing attention-based aggregation module (PMA or Transformer) brings outsized

gains compared to a baseline that does not contextualize the sequence (Bypass). These results are summarized in Table 6.

Table 6 Performance for three aggregation modules. Baseline: model with RH for each module. Semantic ID brings larger gains to the contextualizing modules.

	Train NE Gain	Eval NE Gain
Bypass	-0.056%	-0.085%
Transformer	-0.071%	-0.110%
PMA	-0.073%	-0.100%

To understand how using Semantic ID changes the learned attention patterns in PMA and Transformer aggregation modules, we compute four metrics on the attention scores on a random subset of 1,000 evaluation examples.

Let $\mathbf{A} \in \mathbb{R}^{T \times S}$ be the attention score matrix, where T is the target sequence length and S is the source sequence length. We have $T = S$ for Transformer and Bypass, and $T = 32$ for PMA. Each row $\mathbf{a}_{i,:}$ of $\mathbf{A}$ represents a probability distribution over the source tokens. The metrics we consider are defined as follows.

$$\begin{aligned}
\text{First source token attention: } & \frac{1}{T} \sum_{i=1}^T a_{i,1} \\
\text{Padding token attention: } & \frac{1}{T} \sum_{i=1}^T \sum_{j=1}^S \mathbb{I}\{a_{i,j} = \text{pad}\} \cdot a_{i,j} \\
\text{Entropy: } & \frac{1}{T} \sum_{i=1}^T \sum_{j=1}^S a_{i,j} \cdot \log_2 a_{i,j} \\
\text{Token self-attention: } & \frac{1}{T} \sum_{i=1}^T a_{i,i}
\end{aligned}$$

Table 7 Attention score-based evaluation metrics for random hashing and SemID-based models for the user history item interaction features.

	First	Pad	Entropy	Self
Transformer + RH	0.030	0.460	2.149	0.052
Transformer + SemID	0.043	0.418	1.967	0.045
PMA + RH	0.071	0.351	3.075	-
PMA + SemID	0.074	0.313	3.025	-

From the metric readings in Table 7, we see that models trained with Semantic ID have lower entropy, token self-attention, and padding token attention, and higher attention score on the first source token in the sequence. This means that Semantic ID-based models place more weight on higher-signal tokens (i.e., the first and most recent item in the sequence, rather than earlier and potentially stale tokens or padding tokens), have attention score distributions that are less diffuse over the entire sequence (i.e., lower entropy), and for Transformer, place higher weights on other tokens rather than self-attending. These metrics are promising signals that Semantic ID item representations are more stable and meaningful than their random hashing-based counterparts in user history modeling.

7 Productionization

The Semantic ID features have been productionized in Meta Ads Recommendation System for more than a year. They serve as top sparse features in the existing ads ranking models according to feature importance studies. In this section, we provide an overview of the online serving pipeline and key implementation details.

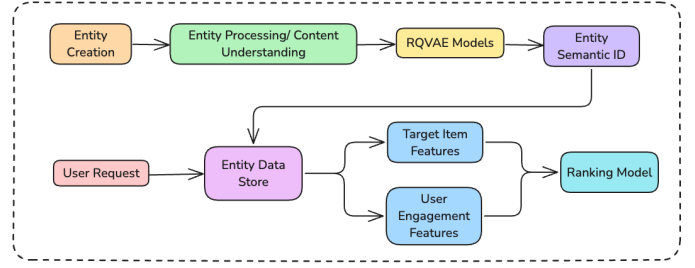


Figure 4 Semantic ID serving pipeline.

7.1 Offline RQ-VAE Training

The RQ-VAE models are trained on Content Understanding (CU) models for ads ranking at Meta. The CU models are pre-trained on the public CC100 dataset [Conneau \(2020\)](#) and then fine-tuned on internal ads datasets. We sample ad IDs and their corresponding content embeddings from the past three months' data and train the RQ-VAE model offline. For production models, we train RQ-VAEs with $L = 6$ and $K = 2048$, and Semantic ID follows the design of prefix-5gram from Section 4.2 with $H = O(50M)$. After training, we use a frozen RQ-VAE checkpoint for online serving.

7.2 Online Semantic ID Serving System

Figure 4 shows the online serving pipeline of real-time Semantic ID features. At ad creation time, we process the ad content information and provide it to the CU models. The output CU embeddings are then passed through the RQ-VAE model which computes the Semantic ID signal for each raw ID. The signal is then stored in the Entity Data Store. At the feature generation stage, the target item raw ID and user engagement raw ID histories are enriched with the Semantic ID signal from the Entity Data Store to produce semantic features. When serving requests arrive, the precomputed features are fetched and passed to the downstream ranking models.

7.3 Production Performance Improvement

We created six sparse and one sequential feature from different content embedding sources, including text, image, and video, and report the NE gain on the flagship Meta ads ranking model in Table 8. In Meta ads ranking, an offline NE gain larger than 0.02% is considered significant. Overall, across multiple ads ranking models, incorporating the Semantic ID features have yielded a **0.15% gain in performance on our top-line online metric**. As the Meta Ads recommender serves billions of users and has been one of the most optimized models in the company, a 0.15% online performance gain is considered significant.

Table 8 NE improvement from incorporating Semantic ID features in the flagship Meta ads ranking model.

	Train NE Gain	Eval NE Gain
Baseline + 6 sparse features	-0.063%	-0.071%
Baseline + 1 sequential feature	-0.110%	-0.123%

7.4 Semantic and Prediction Similarity

Intuitively, one may think that if two items are semantically similar, their user engagement patterns will also be similar. However, user behavior and perceptions are more nuanced and

not predictably continuous with respect to semantics. For robust delivery performance with Semantic ID, we must ensure a degree of continuity (or correlation) of the ranking model’s behaviour with respect to the semantic similarity relation between the items in our system.

To measure this correlation, we conduct an online A/B test where we select a set S of items that are recommended to a user by our system. For a given user, with 50% probability, we mutate the set S to S' by randomly swapping an item in S with a different item with the same prefix from Semantic ID. This operation results in a

$$\text{Click Loss Rate} := \frac{\text{CTR on } S' - \text{CTR on } S}{\text{CTR on } S}. \quad (3)$$

The click loss rate decrease from using deeper prefixes from Semantic ID is summarized in Figure 5.

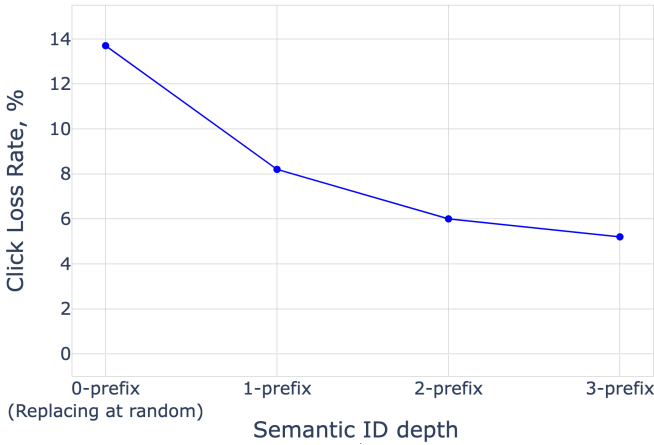


Figure 5 Click Loss Rate reduction from Semantic ID.

Since Semantic ID partitions the item corpus based on item semantics, we conclude that prediction similarity is correlated to semantic similarity. This supports the representation space analysis result in Section 6.3. Moreover, the hierarchy of codes in Semantic ID effectively captures the finer-grained details of an item’s semantics: deeper prefixes monotonically reduce the click loss rate.

7.5 A/A Variance

Yet another disadvantage of a ranking model based on random hashing is inherent model prediction variance, resulting in downstream ad delivery variance. Specifically, one can create a copy of an item with a different raw item ID. Then, both the original and item copy enter the recommendation system. Because the embeddings will be different for the original and item copy after hashing, the model predictions and delivery system behavior will also differ. We call this phenomenon the *A/A variance*, where “A/A” signifies that we consider an exact copy of the original item.¹ This variance is undesirable since it reduces the robustness of the downstream ad ranking orders and the system’s ability to accurately target correct audiences. Semantic ID helps

¹A related type of a ranking model variance is the *A/A’ variance* where one instead considers minor item differences instead of exact copies, such as two images with the same object in the foreground, but with backgrounds of different colors.

reduce the A/A variance by eliminating the randomness caused by random hashing – exact copies or very similar items will often have the same k -prefix Semantic ID.

We set up an online shadow ads experiment where we measure the relative A/A prediction difference (AAR) for a given model. For an A/A pair (a_1, a_2) ,

$$\text{AAR}(a_1, a_2) := 2 \frac{p(a_1) - p(a_2)}{p(a_1) + p(a_2) + \epsilon}, \quad (4)$$

where $p(a_i)$ is the ranking model prediction for item a_i .

The production model with six Semantic ID sparse features achieves a **43% reduction in average AAR** compared to the same model without the six features. We believe that the majority of AAR reduction is from the tail items, as studied in Section 6.2.

8 Conclusion

We show how Semantic ID can be used to create a stable ID space for item representation and propose Semantic ID prefix-gram, which significantly improves Semantic ID’s performance in ranking models. In offline experiments, we study trained ranking models and find that under Semantic ID, the harmful effects of embedding representation instability are mitigated compared to random hashing and individual embeddings baselines. We detail the successful productionization of Semantic ID features in Meta’s ads recommendation system, and show that the online production system obtains significant performance gains as well as reduced downstream ad delivery variance.

References

- Bo Chang, Changping Meng, He Ma, Shuo Chang, Yang Gu, Yajun Peng, Jingchen Feng, Yaping Zhang, Shuchao Bi, Ed H Chi, et al. 2024. Cluster Anchor Regularization to Alleviate Popularity Bias in Recommender Systems. In *Companion Proceedings of the ACM on Web Conference 2024*. 151–160.
- Alexis Conneau. 2019. Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116* (2019).
- Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- Jacob Devlin. 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- João Gama, Indrė Žilobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.
- Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.
- Taku Kudo. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *arXiv preprint arXiv:1808.06226* (2018).

- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*. PMLR, 3744–3753.
- Zhuoran Liu, Leqi Zou, Xuan Zou, Caihua Wang, Biao Zhang, Da Tang, Bolin Zhu, Yijie Zhu, Peng Wu, Ke Wang, and Youlong Cheng. 2022. Monolith: Real Time Recommendation System With Collisionless Embedding Table. [arXiv:2209.07663 \[cs.LG\]](https://arxiv.org/abs/2209.07663) <https://arxiv.org/abs/2209.07663>
- Staša Milojević. 2010. Power law distributions in information science: Making the case for logarithmic binning. *Journal of the American Society for Information Science and Technology* 61, 12 (2010), 2417–2425.
- Maxim Naumov. 2019. On the dimensionality of embeddings for sparse features and data. *arXiv preprint arXiv:1901.02103* (2019).
- Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G Azolini, et al. 2019. Deep learning recommendation model for personalization and recommendation systems. *arXiv preprint arXiv:1906.00091* (2019).
- Haohao Qu, Wenqi Fan, Zihuai Zhao, and Qing Li. 2024. TokenRec: Learning to Tokenize ID for LLM-based Generative Recommendation. *arXiv preprint arXiv:2406.10450* (2024).
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
- Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2024. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2024).
- Rico Sennrich. 2015. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909* (2015).
- Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, et al. 2023. **Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations**. *arXiv preprint arXiv:2306.08121* (2023).
- Ashish Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
- Jingdong Wang, Ting Zhang, Nicu Sebe, Heng Tao Shen, et al. 2017. A survey on learning to hash. *IEEE transactions on pattern analysis and machine intelligence* 40, 4 (2017), 769–790.
- Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. 2009. Feature hashing for large scale multitask learning. In *Proceedings of the 26th annual international conference on machine learning*. 1113–1120.
- Tiansheng Yao, Xinyang Yi, Derek Zhiyuan Cheng, Felix Yu, Ting Chen, Aditya Menon, Lichan Hong, Ed H Chi, Steve Tjoa, Jieqi Kang, et al. 2021. Self-supervised learning for large-scale item recommendations. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 4321–4330.
- Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. 2021. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2021), 495–507.
- Caojin Zhang, Yicun Liu, Yuanpu Xie, Sofia Ira Ktena, Alykhan Tejani, Akshay Gupta, Pranay Kumar Myana, Deepak Dilipkumar, Suvadip Paul, Ikuhiro Ihara, et al. 2020. Model size reduction using frequency based double hashing for recommender systems. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 521–526.

A Aggregation Module Architectures

Bypass. Apply a linear weight matrix $\mathbf{W} \in \mathbb{R}^{d_m \times d_m}$ to each embedding separately,

$$\text{Bypass}(\mathbf{X}) := \mathbf{X}\mathbf{W}. \quad (5)$$

Transformer (Vaswani, 2017). Apply a Transformer layer to the embedding sequence. The attention submodule is defined as

$$\text{Attention}(\mathbf{X}) := \text{softmax} \left(\frac{(\mathbf{X}\mathbf{W}^Q)(\mathbf{X}\mathbf{W}^K)^\top}{\sqrt{d_m}} \right) (\mathbf{X}\mathbf{W}^V), \quad (6)$$

where $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V \in \mathbb{R}^{d_m \times d_a}$ are the query, key, and value weight matrices, respectively, and d_a is the query/key/value vector dimension. The full Transformer module is given by

$$\mathbf{X}^{(1)} = \text{Attention}(\text{LayerNorm}(\mathbf{X})) + \mathbf{X} \quad (7)$$

$$\mathbf{X}^{(2)} = \text{MLP}(\text{LayerNorm}(\mathbf{X}^{(1)})) + \mathbf{X}^{(1)}, \quad (8)$$

where LayerNorm and MLP designate the standard layer norm and position-wise MLP layers. We add standard positional embeddings to the encoding before applying Transformer or PMA modules.

Pooled Multihead Attention (PMA) (Lee et al., 2019). Apply a Transformer layer to the embedding sequence, but replace the attention query vectors with d_s learnable weight vectors. The PMA attention submodule is defined as

$$\text{PMAttention}(\mathbf{X}) := \text{softmax} \left(\frac{\mathbf{S}(\mathbf{X}\mathbf{W}^K)^\top}{\sqrt{d_m}} \right) (\mathbf{X}\mathbf{W}^V), \quad (9)$$

where $\mathbf{S} \in \mathbb{R}^{d_s \times d_a}$ is comprised of d_s learnable query vectors, or seeds. In our experiments, $d_s = 32$.

The PMA module is formed using the same equations as for the Transformer module (Equations 7 and 8), except that PMAttention is used in place of Attention.

B Distributions of Clicks for Semantic ID

The click distribution in Semantic ID space (Figure 6) clearly exhibits less skew compared to the click distribution in the raw ID space. Note that while Figure 2 shows the cumulative distribution of impressions, Figure 6 shows the the marginal distribution of clicks.

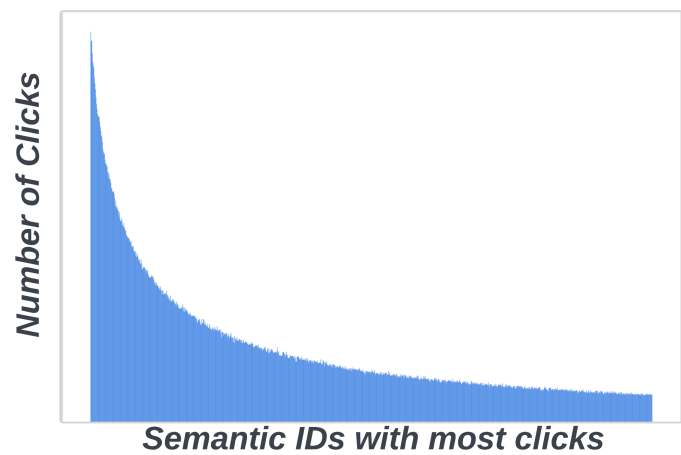
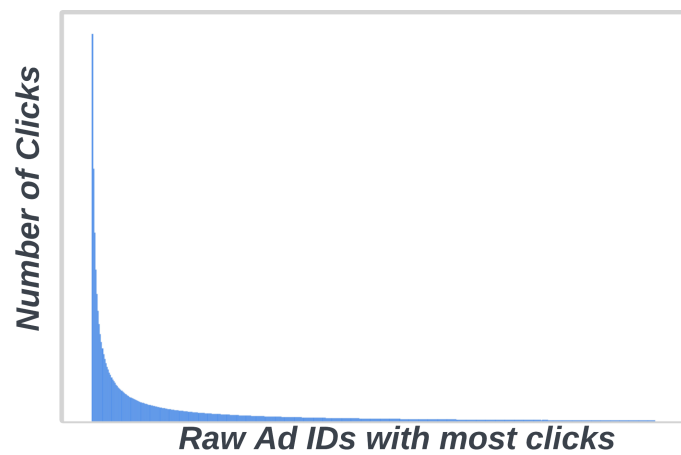


Figure 6 The 30-day click distribution in raw ID and Semantic ID spaces.